

DES Modes of Operation and Brute-Force Cryptanalysis

Mateo Oñate

School of Mathematical and Computational Sciences, Yachay Tech
email: mateo.onate@yachaytech.edu.ec

ABSTRACT We extended our Data Encryption Standard library with electronic codebook and cipher block chaining modes, then compared sequential and parallel key searches. Our mode experiments examined repeated plaintext blocks, changes in the initialization vector, and a single modified ciphertext bit. In our experiment, cipher block chaining produced different ciphertext blocks for the repeated input. Changing one ciphertext bit affected two blocks of the recovered message. We followed the laboratory instructions and searched a space with 16 unknown effective key bits, keeping the other 40 fixed. On an Intel Core i3 laptop, we ran each configuration three times with one, two, and four processes. The average search took 36.063 seconds with one process and 22.006 seconds with four. The highest measured throughput was 2971.4 keys per second, with a speedup of 1.64 and parallel efficiency of 41.0 percent. We also estimated full key space search times from this measured rate. In our measurements, four processes finished sooner, while two processes achieved higher parallel efficiency. The error experiment also showed why valid padding does not establish message authenticity.

INDEX TERMS Block ciphers, brute-force search, cipher modes, parallel computing.

I. Introduction

We added ECB and CBC to our Laboratory 1 DES library and measured a reduced key search, following the two laboratory handouts [1, 2]. We kept the block cipher unchanged. Its permutation tables, S-boxes, and key schedule follow the course slides [3].

We limited the measured search to 16 unknown key bits, following the instructor's scope reported for this laboratory. We compare one, two, and four CPU processes, each repeated three times. DES still has 56 effective key bits [4].

II. Modes Of Operation

DES encrypts one 8-byte block at a time. We added PKCS7 padding before splitting a message into blocks. If r bytes complete the next block, we append r bytes, each with value r . An aligned or empty message receives eight bytes with value 8. This padding rule follows the content-encryption procedure in RFC 5652, Section 6.3 [5].

After decryption, we check the message length, padding count, and every padding byte before removing the suffix. This prevents the decoder from accepting malformed padding.

Electronic codebook (ECB) encrypts each block independently with the same key. For plaintext block P_i , ciphertext block C_i , and key K , encryption follows $C_i = E(K, P_i)$. Decryption follows $P_i = D(K, C_i)$. Equal input blocks produce equal ciphertext blocks under the same key [3].

Cipher block chaining, CBC, combines each plaintext block with the previous ciphertext block through XOR before DES encryption. We define C_0 as the initialization vector, called IV. Encryption follows the equation: $C_i = E(K, P_i \text{ XOR } C_{\text{prev}})$. And the decryption follows the equation: $P_i = D(K, C_i) \text{ XOR } C_{\text{prev}}$.

The IV contains 8 bytes. Our encryption example generates a completely new IV with the secrets module. The caller retains the IV separately from the ciphertext. CBC decryption preserves the original ciphertext block for the next XOR operation.

Both modes call the same DES block functions. We add padding once before encryption and remove it once after recovering the complete message. No external DES implementation performs these operations.

III. ECB Versus CBC Experiment

We repeated the 8-byte string ABCDEFGH four times. The message contains 32 bytes, and padding adds an 8-byte block. We encrypted all five blocks with key 133457799BBCDFF1 and CBC IV 0123456789ABCDEF.

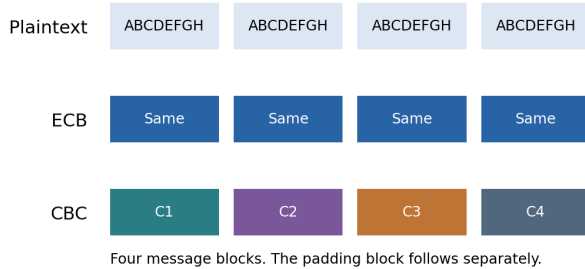


Figure 1: Equal colors indicate equal blocks. ECB preserves the four repeated blocks. CBC produces four different blocks in this experiment.

Table 1: Hexadecimal ciphertext for each block under ECB and CBC. The fifth block corresponds to the padding added before encryption.

Block	ECB	CBC
1	0EE11BD280EF0A1	6732436C6B6D7FC7
2	0EE11BD280EF0A1	8DE3F206177C743E
3	0EE11BD280EF0A1	1884713AB1B23DE1
4	0EE11BD280EF0A1	555F76AC0FA26E67
5	FDF2E174492922F8	D1FCAE0B82B492DB

The four repeated plaintext blocks produced the same ECB value, 0EE11BD280EF0A1. With CBC, each corresponding ciphertext block differed because encryption also depends on the previous ciphertext block. The ECB output preserved the repetition in our message, making the equal blocks identifiable without recovering their contents.

We repeated CBC encryption with IV FEDCBA9876543210 and kept the key and message unchanged. The first ciphertext block changed from 6732436C6B6D7FC7 to C89A8D8E30743B5E. All five ciphertext blocks changed in this test.

IV. Error Propagation

We inverted the first bit of ciphertext block C2 by applying XOR with hexadecimal 80 to byte index 8. We then decrypted with the original key and IV. We kept the final padding block intact during this test.

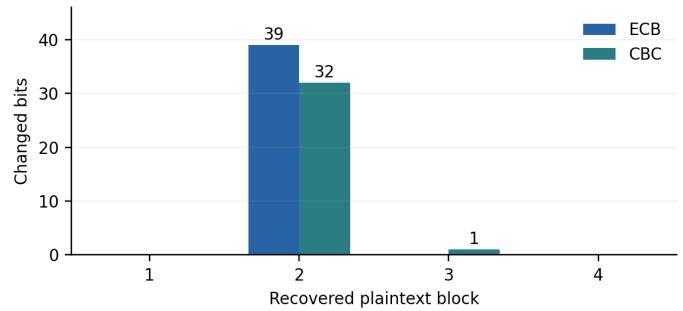


Figure 2: Number of bits differing from the original plaintext after we flipped one bit in ciphertext block C2.

With ECB, the recovered P2 differed from the original in 39 bit positions. P1, P3, and P4 remained unchanged. With CBC, we counted 32 differences in P2 and one in P3. The original P3 value, 4142434445464748, became C142434445464748.

This extra change follows from CBC decryption. To recover P3, we combine $D(K, C3)$ with C2 through XOR. Since C3 stays intact, the flipped bit in C2 changes only the corresponding bit in P3. Later blocks recover normally. The counts of 39 and 32 describe this specific trial, while the single corresponding change in the next CBC block follows directly from XOR. This matches the CBC error behavior in Appendix D of NIST SP 800-38A [6].

V. Brute-Force Method

We fixed the upper 40 effective key bits and varied the lower 16. This produces 65536 candidates, numbered 0 through 65535. The fixed effective key value was 12695BC9B7B7F8 in hexadecimal before clearing its lower 16 bits.

For each candidate, we combine its 16 bits with the fixed portion, divide the 56 effective bits into eight groups of seven, and append one odd parity bit to each group. The resulting 8-byte value represents a DES key. Parity bits do not enlarge the secret search space [4].

The sequential algorithm visits candidates in ascending order, encrypts a known plaintext block, and compares the result with its known ciphertext. It increments the attempt counter after each encryption and returns the first matching candidate. If the interval ends without a match, it returns an unsuccessful result.

Our benchmark placed the target at candidate 65535. The corresponding key was 133457799BBFFEFE. Plaintext 0123456789ABCDEF produced ciphertext 2D7541213B8062E4. The attack receives the known pair and search parameters, without the target key. After the timed search, the benchmark checks the recovered key against a second known pair.

VI. Parallel Implementation

We distributed contiguous intervals among processes through `ProcessPoolExecutor`. Integer division assigns nearly equal ranges without overlaps or gaps. With four processes, each interval contains 16384 candidates.

Table 2: Four-process partition of the 16-bit search. Each upper boundary is exclusive.

Process	First candidate	Upper boundary
1	0	16384
2	16384	32768
3	32768	49152
4	49152	65536

Each worker runs the same search on its assigned interval. A shared event signals a successful match. Workers check this event every 64 attempts. The main process gathers their results, sums actual attempts, and waits for shutdown before stopping the timer.

We selected the `spawn` start method for Windows compatibility and protected process creation with the main entry point guard. A one-process request follows the sequential path and avoids process pool overhead.

VII. Experimental Results

We measured on an Intel Core i3-1005G1, reported at 1.20 GHz, with two physical cores and four logical processors. The environment ran Windows 10 and Python 3.12.7. We completed three repetitions for each process count at 16 bits. Tables 3 and 4 and Figure 3 summarize those nine runs from our submitted measurement files [7]. We retained the original files and selected only their 16-bit records for this report.

The timer records elapsed search time with `perf_counter`. It includes candidate generation, DES encryption, comparison, and process startup and shutdown. We alternate process configurations within each repetition. The setup creates the target pair before timing.

Table 3: Individual execution times in seconds.

Processes	Run 1	Run 2	Run 3
1	35.576	34.242	38.370
2	24.172	24.496	25.505
4	21.967	22.010	22.040

All nine runs recovered candidate 65535. Sequential runs tested 65536 keys each. Four process runs tested 65280, 65344, and 65536 keys. Workers advance at different rates, so the shared stop signal sometimes ends other intervals before completion.

Table 4: Aggregate results. SD denotes sample standard deviation. Efficiency appears as a percentage.

Processes	1	2	4
Mean time, s	36.063	24.725	22.006
SD, s	2.107	0.695	0.036
Keys per second	1817.3	2650.6	2971.4
Speedup	1.000	1.459	1.639
Efficiency, percent	100.0	72.9	41.0

We calculate throughput as total attempts divided by total time. Speedup divides the sequential mean time by the parallel mean time. Efficiency divides speedup by the number of processes. Efficiency describes scaling relative to an ideal reference, rather than CPU utilization.

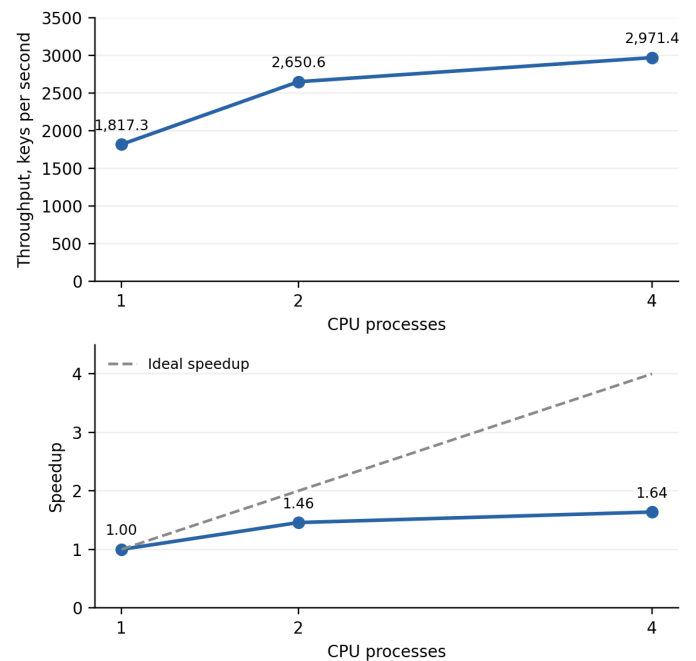


Figure 3: Measured throughput and speedup for 16 bits. Each point summarizes three repetitions. The dashed line represents ideal speedup.

Four processes achieved the shortest mean time, 22.006 seconds, a reduction of 39.0 percent from sequential execution. Two processes achieved 72.9 percent efficiency, compared with 41.0 percent for four. Moving from two to four processes saved another 2.719 seconds on average. The recorded processor has two physical cores, so four processes share physical resources. We did not measure process startup costs separately or record background load. These results cannot assign the smaller gain to one cause.

The project passed 44 automated tests in the preparation environment. The tests cover block cipher correctness, padding, mode round trips, interval partitioning, search outcomes, and metric calculations.

VIII. DES-56 Extrapolation

We projected full DES search time from our best aggregate rate of 2971.374 keys per second. The complete effective key space contains 72057594037927936 candidates, or 2 to the power of 56. We did not execute this full search.

The complete traversal estimate divides the full candidate count by the measured rate. For a uniformly distributed target position, we approximate the expected search time with half the candidate count divided by the same rate.

Table 5: DES-56 time estimates at 2971.374 keys per second. Scientific notation expresses large values.

Unit	Complete traversal	Expected search
Seconds	2.4251e+13	1.2125e+13
Hours	6.7363e+09	3.3681e+09
Days	2.8068e+08	1.4034e+08
Years	7.6845e+05	3.8423e+05

The complete traversal requires approximately 768455 years at this rate. The expected search requires approximately 384227 years. We convert years with 365.25 days per year.

These estimates assume a constant rate, a uniform target position, and no additional key information. They describe our Python implementation on the recorded processor. Dedicated hardware, different implementations, and larger parallel systems require separate performance measurements.

IX. Security Discussion

Our 44 passing tests support the correctness of the tested operations, but DES still has only 56 effective key bits. NIST withdrew FIPS 46-3 in May 2005 [4]. The long projected search time reflects this Python program on our laptop, rather than a claim about DES security against specialized equipment.

The reduced search exposes a practical consequence of partial key knowledge. Fixing 40 effective bits leaves only 65536 candidates. Our sequential mean was about 36 seconds for this search. This result does not demonstrate recovery of an arbitrary full DES key in the same time.

ECB preserves equality between aligned plaintext blocks under one key. In our repeated message, an observer can identify four equal ciphertext blocks without knowing their contents. In our repeated-block experiment, CBC produced different ciphertext blocks through its dependence on the preceding block.

CBC still operates with the same DES key. The public IV changes the input for the first encryption, but it does not enlarge the secret key space. An attacker who knows the first plaintext block and the IV can combine them through XOR, encrypt the result with each candidate key, and compare it with the first ciphertext block.

CBC requires an unpredictable IV for each encryption, as specified in NIST SP 800-38A. This requirement does not provide ciphertext authentication. In our error experiment, decryption accepted the padding even though the recovered message contained altered bytes. Passing the padding check did not estab-

lish message integrity. Applications requiring tamper detection need authentication in addition to encryption.

Parallel execution shortened our reduced search, but four processes did not provide four times the speed. The processor has two physical cores, and the implementation pays coordination costs. Actual attempt counts also differ with early stopping, so time-based speedup reflects both processing rate and completed work.

AES supports 128, 192, and 256-bit keys [8]. For comparison, a 128-bit key space contains 2 to the power of 72 times as many candidates as a 56-bit key space. This ratio is approximately $4.72e21$. The comparison assumes equal candidate testing rates. We did not measure AES encryption or an AES key search.

X. Conclusions

Our ECB and CBC extensions handled arbitrary message lengths through validated PKCS7 padding. The repeated-block experiment produced four equal ECB ciphertext blocks and four different CBC message blocks.

A single modified ciphertext bit can change one recovered block in ECB. In CBC, it can change the corresponding block and one bit in the next. This behavior confirms the different dependencies, showing why valid padding cannot establish authenticity.

All nine 16-bit searches recovered the target key. Four processes reduced mean time from 36.063 to 22.006 seconds, while efficiency fell to 41.0 percent. Our best measured rate projects to roughly 768455 years for complete DES-56 traversal on the same implementation and hardware. The measured gain applies to the nine 16-bit runs. We did not benchmark a full DES key search or specialized hardware.

References

- [1] *Lab 1: Implementation of DES as a Symmetric-Cipher Library*, Cryptography laboratory handout, pp. 1–6.
- [2] *Lab 2: DES Modes of Operation and Brute-Force Cryptanalysis*, Cryptography laboratory handout, pp. 1–14.
- [3] C. Iza, *Data Encryption Standard (DES)*, Cryptography lecture slides, School of Mathematical and Computational Sciences, Yachay Tech University, Sep. 2, 2026, slides 26–47 and 62–74.
- [4] National Institute of Standards and Technology, *Data Encryption Standard (DES)*, FIPS PUB 46-3, Oct. 1999. Withdrawn May 19, 2005. DOI: 10.6028/NIST.FIPS.46-3.
- [5] R. Housley, *Cryptographic Message Syntax (CMS)*, RFC 5652, Sep. 2009, Sec. 6.3. DOI: 10.17487/RFC5652.
- [6] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Methods and Techniques*, NIST SP 800-38A, Dec. 2001, Secs. 5.3, 6.1–6.2, and App. D. DOI: 10.6028/NIST.SP.800-38A.
- [7] M. Onate, *Laboratory 2 DES benchmark measurements*, experimental data, Sep. 9, 2026. Files: benchmark.json, mediciones.csv, and resumen.csv. Analysis subset: 16 unknown bits, three repetitions with 1, 2, and 4 processes.
- [8] National Institute of Standards and Technology, *Advanced Encryption Standard (AES)*, FIPS PUB 197, updated May 9, 2023. DOI: 10.6028/NIST.FIPS.197-upd1.